

브라우저 활동 기반 일기 생성 프로젝트

0. 프로젝트 개요 및 핵심 문제

본 프로젝트는 사용자의 하루 브라우저 활동 기록을 바탕으로 개인화된 일기를 자동 생성하는 시스템을 개발하는 것을 목표로 한다. 실시간 처리는 불필요하며, 하루치 활동을 축적한 뒤 배치 방식으로 처리한다.

핵심 기술 문제 2 가지

- 문제 A — 수집: 사용자가 어떤 페이지를 방문했는지, 얼마나 머물렀는지 수집하는 방법
- 문제 B — 추출: 방문한 페이지에서 핵심 내용을 추출하는 방법 (특히 ChatGPT, Claude 같은 동적 AI 인터페이스 포함)

1. 관련 논문 조사 및 핵심 시사점

1-1. 브라우저 데이터 수집 관련 논문

[논문 1] Web Historian (Menchen-Trevino, 2016)

- 출처: Semantic Scholar — <https://www.semanticscholar.org/paper/Web-historian:-enabling-multi-method-and-research-Menchen-Trevino/5d9c9a6b184af402f2affc058103cf5da4273743>
- 핵심 내용: 기존 웹 브라우징 데이터 연구는 기업 패널에 의존해 재현 가능성이 낮았음. Chrome 브라우저 확장 프로그램이 참여자로부터 안전하게 브라우징 히스토리를 수집하는 방법으로 검증됨.
- 프로젝트 시사점: 크롬 익스텐션 방식이 학술적으로 검증된 수집 방법.

[논문 2] Analysis of Web Browsing Data: A Guide (Clemm von Hohenberg et al., 2024)

- 출처: Social Science Computer Review, DOI: 10.1177/08944393241227868
- 핵심 내용: URL 자체가 방문 콘텐츠에 대한 강력한 단서를 제공함. 도메인 수준에서 분류(소셜 미디어, 뉴스, 개발 등)가 가능하며, 서브도메인이나 경로로 더 세밀한 분류도 가능.

- 프로젝트 시사점: URL + 제목만으로도 일기 소재의 상당 부분을 확보할 수 있음. 본문 추출 없이도 기본 파이프라인 구성 가능.

1-2. 웹 본문 추출 (Boilerplate Removal) 관련 논문

[논문 3] Trafilatura: A Web Scraping Library and Command-Line Tool for Text Discovery and Extraction (Barbaresi, ACL-IJCNLP 2021)

- 출처: ACL Anthology, DOI: 10.18653/v1/2021.acl-demo.15
- 핵심 내용: Python 오픈소스 라이브러리로 본문 텍스트, 댓글, 메타데이터 추출을 지원. HuggingFace, IBM, Microsoft Research 등이 실제 파이프라인에서 사용. 2025 년 기준 GitHub 5,400+ 스타.
- 프로젝트 시사점: 정적 웹페이지에 대해 가장 검증된 오픈소스 추출 도구. `pip install trafilatura` 한 줄로 바로 사용 가능.

[논문 4] HtmlRAG: HTML is Better Than Plain Text for Modeling Retrieved Knowledge in RAG Systems (Tan, Dou et al., 2024)

- 출처: arXiv 2411.02959, 2024 / WWW 2025 게재
- 핵심 내용: HTML 의 계층 구조(h1~h3 제목 태그, 테이블 등)는 의미적 관계를 풍부하게 담고 있어 순수 텍스트보다 LLM 의 맥락 이해에 유리하지만, JS/CSS 등 불필요한 노이즈 코드를 깎아내는 전처리가 필수적임.
- 프로젝트 시사점: HTML 을 무조건 일반 텍스트로 바꾸면 일기 작성 시 정보의 경중(중요 제목 vs 사소한 본문)을 놓침. HTML 의 주요 구조를 보존하면서 노이즈만 날리는 모질라 리더빌리티(@mozilla/readability) 라이브러리 활용이 이 논문이 제시한 전처리(HTML Pruning)의 가장 완벽하고 현실적인 구현 도구임.

[논문 5] An Empirical Comparison of Web Content Extraction Algorithms (Bevendorff et al., SIGIR 2023)

- 출처: ACM SIGIR 2023, DOI: 10.1145/3539618.3591920
- 핵심 내용: 14 개 추출기를 8 개 데이터셋에서 비교. Readability와 Trafilatura가 전반적으로 가장 견고함. 두 도구를 앙상블하면 단일 추출기보다 성능 향상.
- 프로젝트 시사점: Trafilatura(백엔드) + Readability.js(브라우저 내) 조합의 가능성.

[논문 6] Web2Text: Deep Structured Boilerplate Removal (Vogels et al., 2018)

- 출처: ETH Zurich, arXiv:1801.02607
- 핵심 내용: HTML 을 텍스트 블록 단위로 분할한 뒤 CNN+HMM 으로 본문/보일러플레이트를 이진 분류. 광고, 네비게이션, 배너가 하위 애플리케이션 성능에 부정적 영향을 미침을 정량적으로 증명.

- 프로젝트 시사점: 딥러닝 기반으로 소규모 프로젝트에서 직접 대규모 모델의 구현은 과도하다는 것의 증명.

[논문 7] BoilerNet: Boilerplate Removal using Neural Sequence Labeling (Leonhardt et al., WWW 2020)

- 출처: ACM WWW 2020, DOI: 10.1145/3366424.3383547
- 핵심 내용: HTML 태그와 단어만 입력으로 받는 LSTM 기반 시퀀스 레이블링 모델. 수작업 특성 설계 없이 임의 웹페이지의 본문을 브라우저 익스텐션 형태로도 구현 가능.
- 프로젝트 시사점: 구현 복잡도가 높아 직접 사용은 권장하지 않으나, 브라우저 내 딥러닝 본문 추출의 가능성을 보여주는 참고 자료.

Case 1 — URL · 제목 · 체류시간 수집

1-1. 왜 이걸 수집하는가

URL과 탭 제목만으로도 사용자가 오늘 무엇을 했는지 상당 부분 파악할 수 있다. 예를 들어 URL이 `github.com`이면 개발 활동, `youtube.com`이면 영상 시청, `naver.com/news`면 뉴스 소비로 분류된다. 본문 추출보다 훨씬 가볍고 안정적이기 때문에 모든 방문 페이지에 기본으로 적용한다.

1-2. 어떻게 수집하는가

크롬 익스텐션의 백그라운드 스크립트(`background.js`)에서 두 가지 이벤트를 조합해서 사용한다.

이벤트 1 — `chrome.tabs.onActivated`

사용자가 다른 탭으로 이동하거나 새 탭을 열 때 발생하는 이벤트다. 이 시점에 '이전 탭에서 얼마나 머물렀는지' 체류시간을 계산해서 저장한다.

이벤트 2 — `chrome.tabs.onUpdated`

탭 내부에서 URL이 바뀔 때 발생하는 이벤트다. SPA(싱글 페이지 앱)처럼 탭은 유지하면서 페이지만 바뀌는 경우를 잡기 위해 필요하다.

1-3. 노이즈 필터링

아무 방문이나 다 저장하면 일기 소재로 의미 없는 데이터가 쌓인다. 아래와 비슷한 조건을 사전에 규정해 걸러낸다.

필터 조건	이유
체류시간 30초 미만 제외	탭을 열었다가 바로 닫은 경우, 실제로 읽지 않은 것으로 판단
chrome://, about:blank 제외	브라우저 시스템 페이지로 일기 소재 아님
google.com/search 제외	검색 결과 목록 자체는 소재 아님. 클릭해서 들어간 페이지가 소재
동일 URL 반복 방문 시 합산	같은 페이지를 여러 번 방문하면 체류시간을 합산해 1건으로 처리

Case 2 — 정적 페이지 본문 추출 (Readability.js)

2-1. Readability.js 란

Mozilla(Firefox 개발사)가 만든 오픈소스 JavaScript 라이브러리다. Firefox의 '읽기 모드' 버튼을 누르면 화면이 본문만 깔끔하게 남는데, 그 기능을 구현하는 라이브러리가 바로 Readability.js다.

하는 일은 단순하다. HTML 전체에서 광고·네비게이션·사이드바·푸터를 날리고 본문 텍스트만 남긴다. 원리는 DOM 트리에서 텍스트 밀도가 높은 영역을 찾아내는 것이다. 광고나 네비게이션은 텍스트보다 링크·버튼이 많고, 본문은 반대로 텍스트 비중이 높다는 특성을 이용한다.

성능 수치 (Bevendorff et al., SIGIR 2023): 영어 페이지 기준 F1 스코어 약 0.80~0.85. 한국어 페이지에서는 Jung et al.(2021) 기준 최대 40% 낮아질 수 있으므로, 백엔드에서 Trafilatura로 보완한다.

2-2. 왜 서버가 아닌 브라우저 안에서 실행하는가

가장 중요한 이유는 '로그인 세션' 문제다. 서버에서 URL을 재방문하면 사용자의 로그인 쿠키가 없어서 로그인 페이지로 튕기거나 개인화되지 않은 화면이 뜬다. 반면 크롬 익스텐션의 content script는 사용자가 이미 로그인된 브라우저 안에서 실행되므로, 사용자가 실제로 본 화면 그대로에 접근할 수 있다.

- 예시: 네이버 블로그 비공개 글, 회사 내부 Confluence 페이지, Notion 워크스페이스 등
- 서버에서는 접근 불가능하지만, 브라우저 익스텐션에서는 사용자 세션 그대로 접근 가능

2-3. 처리 흐름 단계별 설명

① 탭 이탈 시점 감지

사용자가 탭을 닫거나 다른 탭으로 이동하는 순간을 트리거로 사용한다. 이 시점이 '이 페이지에 대한 방문이 끝났다'는 가장 명확한 신호이기 때문이다.

② content script 에서 Readability.js 실행

content script는 크롬 익스텐션이 각 웹페이지에 주입하는 JavaScript 파일이다. 이 파일 안에 Readability.js를 포함시켜두면, 탭 이탈 시점에 현재 페이지의 DOM을 분석해서 본문 텍스트를 추출한다.

③ 텍스트 트리밍 (500 자 이내)

추출된 본문 전체를 저장하면 데이터 용량이 커지고, 나중에 LLM API에 주입할 때 토큰 비용이 증가한다. 상위 500자만 잘라서 저장한다. 첫 500자에 기사의 핵심 내용이 집중되어 있는 경우가 대부분이기 때문이다.

2-4. 실제 예시

네이버 뉴스 처리 전

[네이버 로고] [검색창] [로그인] [뉴스홈|정치경제사회IT]

삼성전자, 2분기 영업이익 10조 돌파
삼성전자가 2분기 영업이익 10조원을 돌파했다고 밝혔다.
반도체 부문 회복이 주된 원인으로 분석된다...

[관련기사 6개] [댓글 1,204개] [광고배너]
[회사정보 / 이용약관 / 고객센터]

Readability.js 처리 후

삼성전자, 2분기 영업이익 10조 돌파

삼성전자가 2분기 영업이익 10조원을 돌파했다고 밝혔다.
반도체 부문 회복이 주된 원인으로 분석된다...
(본문 텍스트만 계속, 500자 전후에서 트리밍)

1-3. 동적 웹(SPA) 콘텐츠 추출 관련 기술

[논문 8] GCE: Visual Feature 기반 다국어 본문 추출 (Jung et al., Hanyang Univ., 2021)

- 출처: arXiv:2110.14164
- 핵심 내용: Readability.js, DOM Distiller, Web2Text, BoilerNet 4 종 비교 결과, 비영어 데이터셋에서 성능이 최대 40% 하락. 요소의 위치/크기/중앙거리 등 시각적 특성 기반 GCE 방법이 비영어 환경에서 BoilerNet 대비 최대 13% 우수.

- 프로젝트 시사점: 한국어 웹페이지에서 단순 Readability.js는 성능이 낮을 수 있음. Trafilatura가 다국어를 더 잘 지원하므로 백엔드 추출에 Trafilatura 우선 권장.

[기술 표준] MutationObserver API (W3C Living Standard)

- 출처: MDN Web Docs / W3C Living Standard — <https://developer.mozilla.org/ko/docs/Web/API/MutationObserver>
- 핵심 내용: SPA, 무한 스크롤, API 실시간 업데이트 등 동적 콘텐츠 로딩을 감지하는 JavaScript API. 구형 MutationEvent와 달리 변이를 단일 콜백으로 배치 처리하여 성능 영향 최소화.
- 프로젝트 시사점: ChatGPT, Claude 등 생성형 AI 채팅 인터페이스는 모두 SPA로 구현되어 있어 MutationObserver가 핵심 해결책. content script에서 응답 완료 시점을 debounce로 감지하여 텍스트 캡처.

Case 3 — 동적 AI 채팅 화면 (MutationObserver)

3-1. 왜 Readability.js가 안 되는가

Readability.js는 이미 완성된 HTML을 분석하는 도구다. 그런데 ChatGPT, Claude, Gemini 같은 생성형 AI 서비스는 서버에서 완성된 HTML이 오는 게 아니라, JavaScript가 실시간으로 DOM을 계속 조립하는 SPA(Single Page Application) 구조다.

즉 분석할 고정된 HTML 자체가 없다. 사용자가 질문을 보내면 AI가 응답을 글자 단위로 스트리밍하면서 DOM이 수십~수백 번 바뀐다. Readability.js를 실행하는 시점에 따라 빈 화면이 잡히거나 절반만 잡히는 문제가 생긴다.

△ SPA에서 Readability.js를 쓰면 안 되는 이유: 실행 시점에 DOM이 완성되어 있다는 보장이 없다. AI 응답이 스트리밍 중이라면 빈 텍스트나 불완전한 텍스트가 저장된다.

3-2. MutationObserver란

MutationObserver는 DOM이 바뀔 때마다 알려주는 브라우저 내장 JavaScript API다. W3C 표준이라 별도 설치 없이 모든 브라우저에서 동작한다.

특정 DOM 요소를 감시 대상으로 등록해두면, 그 안에서 요소가 추가·삭제·변경될 때마다 콜백 함수를 호출한다. AI 응답 컨테이너를 감시 대상으로 등록하면, 응답 텍스트가 DOM에 추가되는 순간을 정확히 잡아낼 수 있다.

3-3. 스트리밍 완료 시점을 어떻게 판단하는가 (debounce)

AI 응답은 글자 단위로 스트리밍되기 때문에 DOM이 수백 번 바뀐다. 바뀔 때마다 저장하면 동일한 응답이 수백 개 중복 저장된다. 이 문제를 debounce로 해결한다.

debounce란: 이벤트가 연속으로 발생할 때, 마지막 이벤트 후 일정 시간이 지나야 실제 처리를 실행하는 기법. 중간에 이벤트가 또 오면 타이머를 초기화한다.

AI 응답 스트리밍 중에는 DOM이 계속 바뀌므로 타이머가 계속 초기화된다. 응답이 완전히 끝나면 800ms 동안 DOM 변화가 없어지고, 그때 비로소 저장에 실행된다.

3-4. 실제 처리 흐름 단계별 설명

Claude 화면 예시

```
[사용자] 리액트 훅 설명해줘
[Claude]  ← DOM: 빈 div만 존재
[Claude] React Hook은... ← DOM 변화 (1회)
[Claude] React Hook은 함수형... ← DOM 변화 (2회)
[Claude] React Hook은 함수형 컴포넌트에서... ← DOM 변화 (3회)
... (수십~수백 회 반복)
[Claude] React Hook은 함수형 컴포넌트에서 ← 스트리밍 완료
상태를 관리할 수 있게 해주는 기능...
```

MutationObserver + debounce 처리 흐름

```
DOM 변화 감지 → 타이머 시작(800ms)
DOM 변화 감지 → 타이머 초기화 후 재시작(800ms) ← 스트리밍 진행 중
DOM 변화 감지 → 타이머 초기화 후 재시작(800ms) ← 계속 진행 중
...
DOM 변화 없음 → 800ms 경과
→ '응답 완료' 판단 ☒
→ 최종 텍스트 한 번만 저장
```

2. 현실적 아키텍처 — 선 축적 후 배치 처리

전체 흐름 요약

[Chrome Extension] → [로컬 저장소 DB] → [Python 배치 백엔드] → [일기 생성 LLM API]

Step 1 — 데이터 수집 (Chrome Extension)

1-A. URL·탭 제목·체류 시간 수집

chrome.history API 또는 chrome.tabs.onUpdated 이벤트로 방문 URL, 탭 제목, 타임스탬프를 수집한다. 단순 탭 전환 노이즈를 제거하기 위해 체류 시간 30초 이상인 방문만 저장하도록 필터링한다.

- 트리거: visibilitychange 이벤트 + 타이머(30 초 이상 체류 확인)
- 저장 데이터: { url, title, visitTime, durationSec }
- 논문 근거: Menchen-Trevino(2016), Clemm von Hohenberg(2024)

1-B. 정적 페이지 본문 추출 (Readability.js)

Mozilla Readability.js를 익스텐션 content script에 내장하여, 탭 이탈 시점에 즉시 실행한다. 광고·네비게이션 바를 제거한 순수 본문 텍스트를 500자 이내로 트리밍하여 저장한다. 이를 통해 백엔드 재방문 없이 개인화된(로그인 세션 포함) 페이지의 본문을 캡처할 수 있다.

- 적용 대상: 일반 뉴스, 블로그, 문서, GitHub 등 정적/SSR 페이지
- 저장 데이터: { ...기존 메타데이터, bodyText(max 500 자) }
- 논문 근거: Bevendorff et al. SIGIR 2023 — Readability 성능 검증

1-C. 동적 AI 채팅 페이지 처리 (MutationObserver)

ChatGPT, Claude, Gemini 등 생성형 AI 서비스는 SPA 구조로 동작하여 Readability.js로 본문 추출이 불가능하다. content script에 MutationObserver를 삽입하여 AI 응답이 DOM에 추가되는 시점을 감지하고, debounce(800ms)로 스트리밍 완료 후 최종 텍스트만 저장한다.

- 트리거: MutationObserver 가 [data-message-author-role] 등 응답 컨테이너 요소의 추가를 감지
- debounce 800ms 적용 → 스트리밍 중 중간 텍스트 제외, 완성된 응답만 캡처
- 저장 형태: { type: 'ai_chat', platform: 'claude.ai', content: '응답 텍스트', time: ... }

1-D. 컨텍스트 길이 제어 (청킹 전략)

하루 수십 개 사이트의 본문을 모두 LLM에 주입하면 토큰 비용이 폭증한다. 다음과 같은 규칙으로 사전에 제어하는 것을 고려해볼 수 있다.

- 동일 도메인 반복 방문 → 가장 긴 체류 1 건만 유지, 나머지 제목만 보존
- 본문 텍스트 상한: 일반 페이지 500 자, AI 채팅 응답 800 자

3. 참고문헌

1. Menchen-Trevino, E. (2016). Web Historian: Enabling Multi-method and Independent Research with Real-World Web Browsing History Data. Semantic Scholar.
 2. Clemm von Hohenberg, B. et al. (2024). Analysis of Web Browsing Data: A Guide. Social Science Computer Review. DOI: 10.1177/08944393241227868
 3. Barbaresi, A. (2021). Trafilatura: A Web Scraping Library and Command-Line Tool for Text Discovery and Extraction. ACL-IJCNLP 2021 System Demonstrations, pp. 122-131. DOI: 10.18653/v1/2021.acl-demo.15
 4. Bevendorff, J., Gupta, S., Kiesel, J., & Stein, B. (2023). An Empirical Comparison of Web Content Extraction Algorithms. SIGIR 2023. DOI: 10.1145/3539618.3591920
 5. Vogels, T., Ganea, O., & Eickhoff, C. (2018). Web2Text: Deep Structured Boilerplate Removal. arXiv:1801.02607
 6. Leonhardt, J., Anand, A., & Khosla, M. (2020). Boilerplate Removal using a Neural Sequence Labeling Model. WWW '20 Companion. DOI: 10.1145/3366424.3383547
 7. Jung, G. et al. (2021). Don't Read, Just Look: Main Content Extraction from Web Pages Using Visual Features. Hanyang University. arXiv:2110.14164
 8. W3C. MutationObserver API. W3C Living Standard.
<https://developer.mozilla.org/ko/docs/Web/API/MutationObserver>
-